



10.3.5 The Francis implicit QR Step ¶

10.3.5 Implicitly shifted QR algorithm

fit width



In the last unit, we described how, when $A^{(k)}$ is tridiagonal, the steps

$$A^{(k)} \rightarrow Q^{(k)} R^{(k)}$$

$$A^{(k+1)} := R^{(k)} Q^{(k)}$$

of an unshifted QR algorithm can be staged as the computation and application of a sequence of Givens' rotations. Obviously, one could explicitly form $A^{(k)} - \mu_k I$, perform these computations with the resulting matrix, and then add $\mu_k I$ to the result to compute

$$A^{(k)} - \mu_k I \rightarrow Q^{(k)} R^{(k)}$$

$$A^{(k+1)} := R^{(k)} Q^{(k)} + \mu_k I.$$

The Francis QR Step combines these separate steps into a single one, in the process casting all computations in terms of unitary similarity transformations, which ensures numerical stability.

Consider the 4×4 tridiagonal matrix

$$\begin{pmatrix} \alpha_{0,0} & \alpha_{0,1} & 0 & 0 \\ \alpha_{1,0} & \alpha_{1,1} & \alpha_{1,2} & 0 \\ 0 & \alpha_{2,1} & \alpha_{2,2} & \alpha_{2,3} \\ 0 & 0 & \alpha_{3,2} & \alpha_{3,3} \end{pmatrix} - \mu I$$

The first Givens' rotation is computed from $\begin{pmatrix} \alpha_{0,0} - \mu \\ \alpha_{1,0} \end{pmatrix}$, yielding $\gamma_{1,0}$ and $\sigma_{1,0}$ so that

$$\begin{pmatrix} \gamma_{1,0} & -\sigma_{1,0} \\ \sigma_{1,0} & \gamma_{1,0} \end{pmatrix}^T \begin{pmatrix} \alpha_{0,0} - \mu \\ \alpha_{1,0} \end{pmatrix}$$

has a zero second entry. Now, to preserve eigenvalues, any orthogonal matrix that is applied from the left must also have its transpose applied from the right. Let us compute

$$\begin{pmatrix} \alpha_{0,0} & \alpha_{1,0} & \alpha_{2,0} & 0 \\ \alpha_{1,0} & \alpha_{1,1} & \alpha_{1,2} & 0 \\ \alpha_{2,0} & \alpha_{2,1} & \alpha_{2,2} & \alpha_{2,3} \\ 0 & 0 & \alpha_{3,2} & \alpha_{3,3} \end{pmatrix} = \begin{pmatrix} \gamma_{1,0} & \sigma_{1,0} & 0 & 0 \\ -\sigma_{1,0} & \gamma_{1,0} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \alpha_{0,0} & \alpha_{0,1} & 0 & 0 \\ \alpha_{1,0} & \alpha_{1,1} & \alpha_{1,2} & 0 \\ 0 & \alpha_{2,1} & \alpha_{2,2} & \alpha_{2,3} \\ 0 & 0 & \alpha_{3,2} & \alpha_{3,3} \end{pmatrix} \begin{pmatrix} \gamma_{1,0} & -\sigma_{1,0} & 0 & 0 \\ \sigma_{1,0} & \gamma_{1,0} & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

This is known as "introducing the bulge."

Next, from $\begin{pmatrix} \hat{\alpha}_{1,0} \\ \hat{\alpha}_{2,0} \end{pmatrix}$, one can compute $\gamma_{2,0}$ and $\sigma_{2,0}$ so that

$$\begin{pmatrix} \gamma_{2,0} & -\sigma_{2,0} \\ \sigma_{2,0} & \gamma_{2,0} \end{pmatrix}^T \begin{pmatrix} \hat{\alpha}_{1,0} \\ \hat{\alpha}_{2,0} \end{pmatrix} = \begin{pmatrix} \tilde{\alpha}_{1,0} \\ 0 \end{pmatrix}.$$

Then

$$\begin{pmatrix} \tilde{\alpha}_{0,0} & \tilde{\alpha}_{1,0} & 0 & 0 \\ \tilde{\alpha}_{1,0} & \tilde{\alpha}_{1,1} & \hat{\alpha}_{2,1} & \hat{\alpha}_{3,1} \\ 0 & \hat{\alpha}_{2,1} & \hat{\alpha}_{2,2} & \hat{\alpha}_{2,3} \\ 0 & \hat{\alpha}_{3,1} & \hat{\alpha}_{3,2} & \alpha_{3,3} \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \gamma_{2,0} & \sigma_{2,0} & 0 \\ 0 & -\sigma_{2,0} & \gamma_{2,0} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \tilde{\alpha}_{0,0} & \tilde{\alpha}_{1,0} & \hat{\alpha}_{2,0} & 0 \\ \hat{\alpha}_{1,0} & \hat{\alpha}_{1,1} & \hat{\alpha}_{1,2} & 0 \\ \hat{\alpha}_{2,0} & \hat{\alpha}_{2,1} & \alpha_{2,2} & \alpha_{2,3} \\ 0 & 0 & \alpha_{3,2} & \alpha_{3,3} \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \gamma_{2,0} & -\sigma_{2,0} & 0 \\ 0 & \sigma_{2,0} & \gamma_{2,0} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

again preserves eigenvalues. Finally, from

$$\begin{pmatrix} \hat{\alpha}_{2,1} \\ \hat{\alpha}_{3,1} \end{pmatrix},$$

one can compute $\gamma_{3,1}$ and $\sigma_{3,1}$ so that

$$\begin{pmatrix} \gamma_{3,1} & -\sigma_{3,1} \\ \sigma_{3,1} & \gamma_{3,1} \end{pmatrix}^T \begin{pmatrix} \hat{\alpha}_{2,1} \\ \hat{\alpha}_{3,1} \end{pmatrix} = \begin{pmatrix} \tilde{\alpha}_{2,1} \\ 0 \end{pmatrix}.$$

Then

$$\begin{pmatrix} \tilde{\alpha}_{0,0} & \tilde{\alpha}_{1,0} & 0 & 0 \\ \tilde{\alpha}_{1,0} & \tilde{\alpha}_{1,1} & \tilde{\alpha}_{2,1} & 0 \\ 0 & \tilde{\alpha}_{2,1} & \tilde{\alpha}_{2,2} & \tilde{\alpha}_{2,3} \\ 0 & 0 & \tilde{\alpha}_{3,2} & \tilde{\alpha}_{3,3} \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \gamma_{3,2} & \sigma_{3,2} \\ 0 & 0 & -\sigma_{3,2} & \gamma_{3,2} \end{pmatrix} \begin{pmatrix} \tilde{\alpha}_{0,0} & \tilde{\alpha}_{1,0} & 0 & 0 \\ \tilde{\alpha}_{1,0} & \tilde{\alpha}_{1,1} & \hat{\alpha}_{2,1} & \hat{\alpha}_{3,1} \\ 0 & \hat{\alpha}_{2,1} & \hat{\alpha}_{2,2} & \hat{\alpha}_{2,3} \\ 0 & \hat{\alpha}_{3,1} & \hat{\alpha}_{3,2} & \alpha_{3,3} \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \gamma_{3,1} & -\sigma_{3,1} \\ 0 & 0 & \sigma_{3,1} & \gamma_{3,1} \end{pmatrix},$$

yielding a tridiagonal matrix. The process of transforming the matrix that results from introducing the bulge (the nonzero element $\hat{\alpha}_{2,0}$) back into a tridiagonal matrix is commonly referred to as "chasing the bulge." Moving the bulge one row and column down the matrix is illustrated in [Figure 10.3.5.1](#). The process of determining the first Givens' rotation, introducing the bulge, and chasing the bulge is known as a Francis Implicit QR step. An algorithm for this is given in [Figure 10.3.5.2](#).

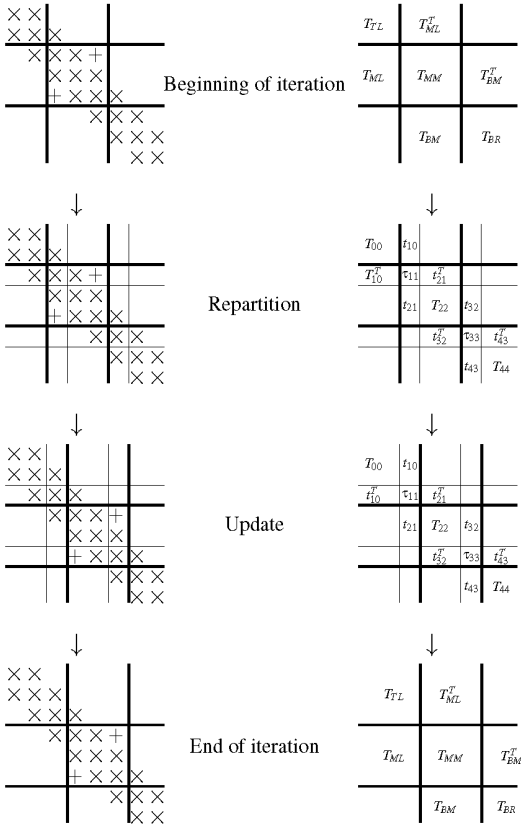


Figure 10.3.5.1. Illustration of how the bulge is chased one row and column forward in the matrix.

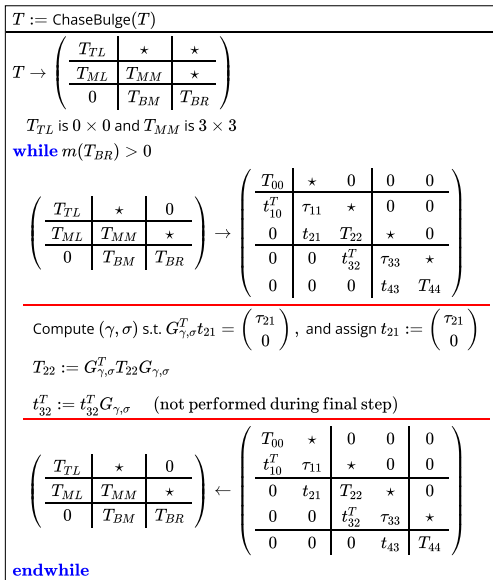


Figure 10.3.5.2. Algorithm for "chasing the bulge" that, given a tridiagonal matrix with an additional nonzero $\alpha_{2,0}$ element, reduces the given matrix back to a

tridiagonal matrix.

The described process has the net result of updating $A^{(k+1)} = Q^T A^{(k)} Q^{(k)}$, where Q is the orthogonal matrix that results from multiplying the different Givens' rotations together:

$$Q = \left(\begin{array}{ccc|cc} \gamma_{1,0} & -\sigma_{1,0} & 0 & 0 & \\ \sigma_{1,0} & \gamma_{1,0} & 0 & 0 & \\ \hline 0 & 0 & 1 & 0 & \\ 0 & 0 & 0 & 1 & \end{array} \right) \left(\begin{array}{ccc|cc} 1 & 0 & 0 & 0 & \\ 0 & \gamma_{2,0} & -\sigma_{2,0} & 0 & \\ \hline 0 & \sigma_{2,0} & \gamma_{2,0} & 0 & \\ 0 & 0 & 0 & 1 & \end{array} \right) \left(\begin{array}{ccc|cc} 1 & 0 & 0 & 0 & \\ 0 & 1 & 0 & 0 & \\ \hline 0 & 0 & \gamma_{3,1} & -\sigma_{3,1} & \\ 0 & 0 & \sigma_{3,1} & \gamma_{3,1} & \end{array} \right).$$

Importantly, the first column of Q , given by

$$\begin{pmatrix} \gamma_{1,0} \\ \sigma_{1,0} \\ 0 \\ 0 \end{pmatrix},$$

is exactly the same first column had Q been computed as in [Subsection 10.3.3 \(10.3.1\)](#). Thus, by the Implicit Q Theorem, the tridiagonal matrix that results from this approach is equal to the tridiagonal matrix that would be computed by applying the QR factorization from that section to $A - \mu I$, $A - \mu I \rightarrow QR$ followed by the formation of $RQ + \mu I$ using the algorithm for computing RQ in [Subsection 10.3.3](#).

Remark 10.3.5.3. In [Figure 10.3.5.2](#), we use a variation of the notation we have encountered when presenting many of our algorithms, including most recently the reduction to tridiagonal form. The fact is that when implementing the implicitly shifted QR algorithm, it is best to do so by explicitly indexing into the matrix. This tridiagonal matrix is typically stored as just two vectors: one for the diagonal and one for the subdiagonal.

Homework 10.3.5.1. A typical step when "chasing the bulge" one row and column further down the matrix involves the computation

$$\left(\begin{array}{ccc|cc} \alpha_{i-1,i-1} & \times & \times & 0 & \\ \hat{\alpha}_{i,i-1} & \hat{\alpha}_{i,i} & \times & 0 & \\ \hline 0 & \hat{\alpha}_{i+1,i} & \hat{\alpha}_{i+1,i+1} & \times & \\ 0 & \hat{\alpha}_{i+2,i} & \hat{\alpha}_{i+2,i+1} & \alpha_{i+2,i+2} & \end{array} \right) = \left(\begin{array}{ccc|cc} 1 & 0 & 0 & 0 & \\ 0 & \gamma_i & \sigma_i & 0 & \\ \hline 0 & -\sigma_i & \gamma_i & 0 & \\ 0 & 0 & 0 & 1 & \end{array} \right) \left(\begin{array}{ccc|cc} \alpha_{i-1,i-1} & \times & \times & 0 & \\ \alpha_{i,i-1} & \alpha_{i,i} & \times & 0 & \\ \hline \alpha_{i+1,i-1} & \alpha_{i+1,i} & \alpha_{i+1,i+1} & \times & \\ 0 & 0 & \alpha_{i+2,i+1} & \alpha_{i+2,i+2} & \end{array} \right) \left(\begin{array}{ccc|cc} 1 & 0 & 0 & 0 & \\ 0 & \gamma_i & -\sigma_i & 0 & \\ \hline 0 & \sigma_i & \gamma_i & 0 & \\ 0 & 0 & 0 & 1 & \end{array} \right)$$

Give a strategy (or formula) for computing

$$\left(\begin{array}{ccc|cc} & & & & \\ \hline \hat{\alpha}_{i,i-1} & \hat{\alpha}_{i,i} & & & \\ \hline \hat{\alpha}_{i+1,i} & \hat{\alpha}_{i+1,i+1} & & & \\ \hline \hat{\alpha}_{i+2,i} & \hat{\alpha}_{i+2,i+1} & & & \end{array} \right)$$

▼ [Solution](#)

Since the subscripts will drive us crazy, let's relabel, add one of the entries above the diagonal, and drop the subscripts on γ and σ :

$$\left(\begin{array}{ccc|cc} \times & \times & \times & 0 & \\ \hat{\epsilon} & \hat{\kappa} & \hat{\lambda} & 0 & \\ \hline 0 & \hat{\lambda} & \hat{\mu} & \times & \\ 0 & \hat{\chi} & \hat{\psi} & \times & \end{array} \right) = \left(\begin{array}{ccc|cc} 1 & 0 & 0 & 0 & \\ 0 & \gamma & \sigma & 0 & \\ \hline 0 & -\sigma & \gamma & 0 & \\ 0 & 0 & 0 & 1 & \end{array} \right) \left(\begin{array}{ccc|cc} \times & \times & \times & 0 & \\ \epsilon & \kappa & \lambda & 0 & \\ \hline \phi & \lambda & \mu & \times & \\ 0 & 0 & \psi & \times & \end{array} \right) \left(\begin{array}{ccc|cc} 1 & 0 & 0 & 0 & \\ 0 & \gamma & -\sigma & 0 & \\ \hline 0 & \sigma & \gamma & 0 & \\ 0 & 0 & 0 & 1 & \end{array} \right)$$

With this, the way I would compute the desired results is via the steps

- $\hat{\epsilon} := \gamma\epsilon + \sigma\phi$
- $\begin{pmatrix} \hat{\kappa} & \hat{\lambda} \\ \hat{\lambda} & \hat{\mu} \end{pmatrix} := \left[\begin{pmatrix} \gamma & \sigma \\ -\sigma & \gamma \end{pmatrix} \begin{pmatrix} \kappa & \lambda \\ \lambda & \mu \end{pmatrix} \right] \begin{pmatrix} \gamma & -\sigma \\ \sigma & \gamma \end{pmatrix}$
- $\hat{\chi} := \sigma\psi$
- $\hat{\psi} := \gamma\psi$

Translating this to the update of the actual entries is straight forward.

Ponder This 10.3.5.2. Write a routine that performs one Francis implicit QR step. Use it to write an implicitly shifted QR algorithm.